Design Artifact - Chronicles of Change

Architectural Style - Layered Architecture

We chose a Layered Architecture for our project Chronicles of Change. The system is split into three separate layers and each layer has one clear job. The three layers are the Presentation Layer, the Application Logic Layer, and the Data Layer. Each layer only talks to the layer directly below it, which means the Presentation Layer talks to the Application Logic Layer, and the Application Logic Layer talks to the Data Layer. No layer skips over another.

We chose this because it gives our group a clean separation between the parts of the system. The person working on the game visuals does not have to touch the database code, and the person working on the database does not have to touch the game engine code. This makes the system much easier to work on, test, and update. If something needs to change in how questions are stored in the database, that change does not break the game UI or the combat logic. Each part of the system can be changed, tested, and improved on its own without affecting the rest of the system. This also makes bugs easier to find because each layer has a clear and limited responsibility, so if something goes wrong it is easier to know which layer the problem is in.

The Three Layers

1. Presentation Layer

This layer is mainly responsible for everything the player and the admin see and interact with. It handles display and input and passes everything down to the layer below it. It has two parts.

The first part is the Game Client built in Ren'Py. It includes the main menu, the combat screen, the quiz screens, the dialogue display, the HUD which shows score, health, and timer, the modifier choice and penalty display, and the save and load preferences screen.

The second part is the Dashboard built in Next.js and React. It includes campaign management, stage management, a question editor, an enemy editor, a dialogue editor, and a tile map editor. The dashboard is how the admin manages all the game content without touching the code.

2. Application Logic Layer

This layer handles all the logic of the game. It has two parts. The first part is the Game Engine built in Python and Ren'Py. It includes the game flow control and update loop in script.rpy, the vector, sprite, and collision handling systems in core.rpy, the player movement and attacks in player.rpy, the enemy behaviour in enemies.rpy, the combat stage that puts everything together in combat_stage.rpy, the quiz system that handles questions and score calculation in quiz_system.rpy, the modifier system that handles rewards and penalties in modifier_system.rpy, the stage configs that define each level in stage_configs.rpy, the dialogue scripts that handle the narrative in dialogue_scripts.rpy, and the interactive items such as portals and other stage objects in items.rpy., the quiz system that handles questions and score calculation in quiz_system.rpy, the modifier system that handles rewards and penalties in modifier_system.rpy, the stage configs that define each level in stage_configs.rpy, the

dialogue scripts that handle the narrative in dialogue_scripts.rpy, and the interactive items including RubyMaker, Ruby, and Portal in items.rpy.

The second part is the Config Loader and the Dashboard Backend API. The Config loader fetches content from the Dashboard API, caches it locally, and parses it as JSON. The dashboard backend API is a Next.js route handler that serves the content to the game from the database.

3. Data Layer

This layer handles all data storage. It has four parts.

The first part is the PostgreSQL Database managed through Prisma ORM. This is the main source of all game content and it stores tables for campaigns, stages, characters, dialogue lines, enemy templates, stage enemies, questions, modifier definitions, modifier pools, player configs, sprite assets, and tile types.

The second part is the Local Cache which is a JSON file stored on the device. If the Dashboard API is not available, the game loads content from this file instead.

The third part is the Ren'Py Persistent Storage which saves the player's high score and carries it across game sessions.

The fourth part is the Hardcoded Defaults which are values already written inside the game code. If both the API and the local cache are unavailable, the game uses these so it can still run.

<u>How the Layers Communicate</u>

The Ren'Py game client sends a request to the config loader in the application logic layer. The config loader first attempts to load content from the locally cached JSON file. If cached data is not available, it requests the content from the dashboard backend API. The dashboard backend API queries the PostgreSQL database and returns the content. If both the cached file and the API are unavailable, the game falls back to the hardcoded defaults already in the code. The content then travels back up to the game client, which displays it to the player. If the API is not available, the config loader uses the locally cached JSON file instead. If that is also not available, the game falls back to the hardcoded defaults already in the code. The content then travels back up to the game client which displays it to the player.

The Next.js dashboard communicates directly with the Dashboard Backend API so that admins can add, edit, and delete game content from a browser without touching any code.

Diagram 1: Architecture Diagram (Component diagram)

The architecture diagram below shows the three layers, the components inside each layer, and how they communicate. It also shows the three-step fallback system in the config loader. The game first tries the local cache, then the Dashboard API, and then the hardcoded defaults.

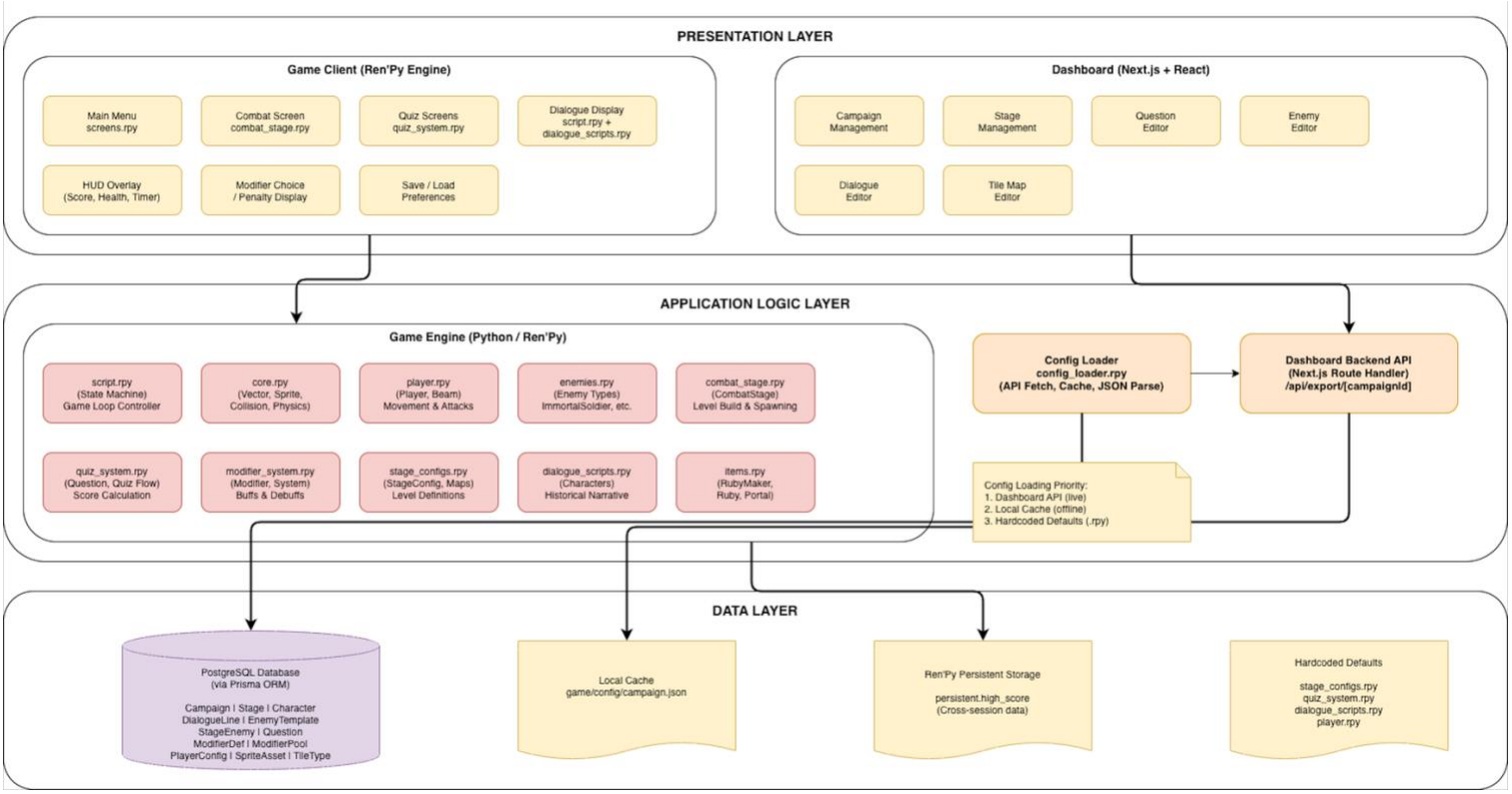
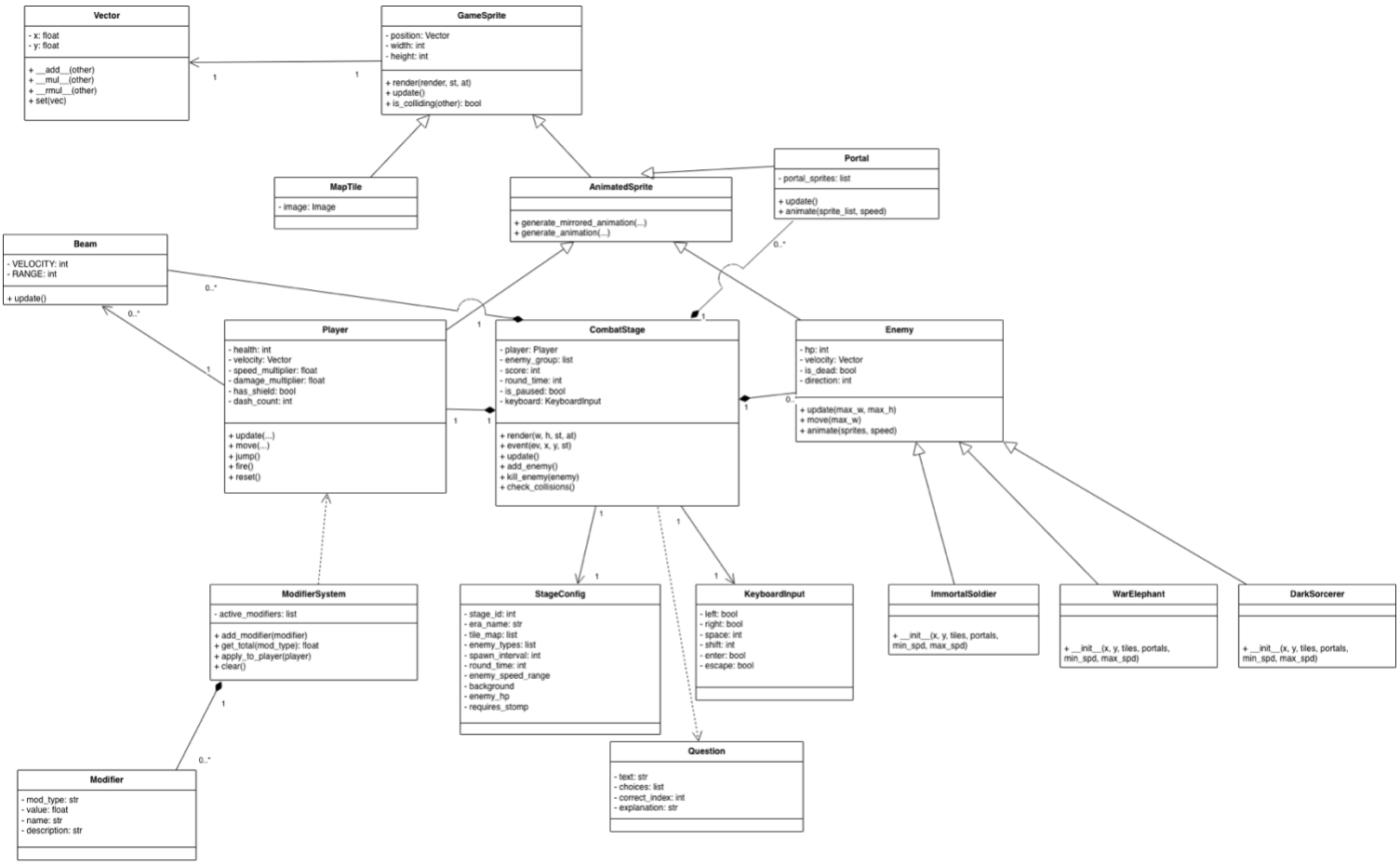


Diagram 2 : UML Class Diagram

The UML class diagram below shows all the major classes in the game engine and how they relate to each other.



<u>Below are each class info:</u>

CombatStage - The central class that runs the combat loop. It holds the Player, the list of enemies, the score, the round timer, and the keyboard input. It handles rendering, events, updating the game state, adding, and removing enemies, and checking collisions.

Player - Represents the player character. It stores health, velocity, speed multiplier, damage multiplier, whether the player has a shield, and the dash count. It handles movement, jumping, firing beams, and resetting after death.

Enemy - The base class used for enemies in the combat system. It stores HP, velocity, alive or dead status, and direction. Different enemy types are configured through stage data, such as speed, health, and behaviour parameters, rather than separate subclass implementations.

GameSprite - The base class for all visual objects in the game. It stores position, width, and height, and it handles rendering, updating, and collision detection.

AnimatedSprite - Extends GameSprite and adds the ability to generate animations for game objects that need them.

ModifierSystem - Manages all active modifiers on the player. It can add a modifier, get the total value by type, apply all active modifiers to the player, and clear them all.

Modifier - Represents one individual modifier. It stores the modifier type, value, name, and description.

StageConfig - Stores all the configuration for one stage, including the stage ID, era name, tile map, enemy types, spawn interval, round time, enemy speed and HP ranges, background, and whether the stomp mechanic is required.

Question - Stores one quiz question. It has the question text, the list of answer choices, the index of the correct answer, and a historical explanation shown after the player answers.

KeyboardInput - Tracks the current state of all keys the game uses - left, right, space, shift, enter, and escape.

Beam - A projectile the player fires (character's weapon). It has a fixed velocity and a maximum range.

MapTile - Represents one tile on the stage map and stores its image.

Portal - A special interactive object used in stages. It stores portal sprites and handles its own animation and movement.

Vector - A 2D math utility used to represent positions and velocities across the entire game.